

Process, Teams, Debt, and Responsible Engineering

Module 2, Lectures 7 and 8 in one sitting, then the Midterm 2 review

CS 5090 · Software Engineering with Generative AI · Week 12 · Module 2, Lectures 7–8 and review

Michigan Technological University · Fall 2026

Two processes, one project

Both are rational. They pull in opposite directions

- Data science: exploratory, iterative, outcome uncertain (explore → maybe).
- Software engineering: planned, incremental, outcome specified (plan → deliver).
- You cannot schedule a discovery, and you cannot ship an experiment.
- Resolution: timebox the exploring and treat its output as a decision, not a deliverable. "Two weeks to find out whether this is feasible" is schedulable; "hit 95%" is not.

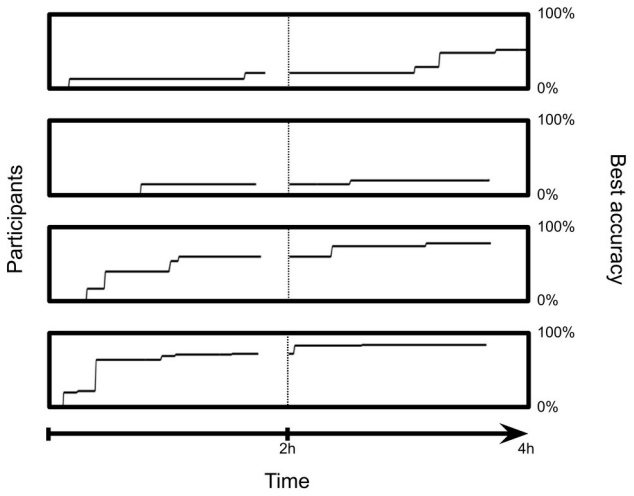


Figure: Kästner, Machine Learning in Production, MIT Press 2025, ch. 20 (CC BY-NC-ND 4.0). Amershi et al. 2019.

Teams: unicorns are rare, T-shaped people are the goal

Coordination cost comes from handoffs, not headcount

- Nobody is expert in ML, product, infrastructure, security, and the domain at once.
- T-shaped: deep in one area, literate in the neighbouring ones. This course is building that shape: Module 1 depth in models, Module 2 literacy in systems, Module 3 in practice.
- Conflicting incentives are the usual root of friction; groupthink is the failure of a team that agrees too easily.

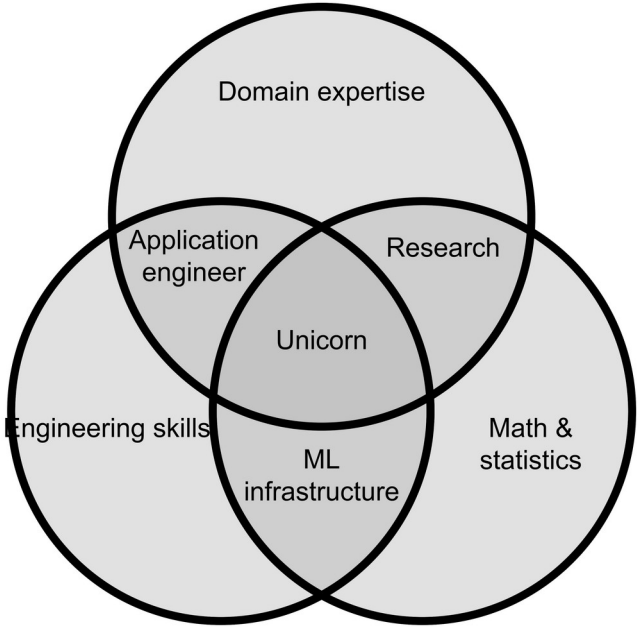


Figure: Kästner 2025, ch. 21. Amershi et al. 2019.

Technical debt, ML edition

Five kinds you will not have met in a normal codebase

Kind	What it looks like
Glue code	95% of the repo is plumbing around a tiny model call
Configuration debt	hundreds of untested, undocumented knobs
Undeclared consumers	someone is reading your output and you do not know
Entanglement	CACE: Changing Anything Changes Everything
Data dependencies	an upstream feed changes format and your model degrades

CACE, concretely: add one feature and every weight shifts; retrain and the whole boundary moves; downstream thresholds misfire. "Small model change" has no meaning, so regression-test the whole system.



Managing the debt, and debt at machine speed

Deliberate and prudent debt is engineering. The other three quadrants need an exit plan

- Deliberate and prudent: written down, with a date to pay it back. Reckless or accidental: the quadrants teams live in without noticing.
- New since five years ago: generated code arrives faster than it can be understood; reviewers approve volume they would have questioned; nobody can explain why a block is the way it is. Debt accumulates at machine speed.
- ⚙️ Every easy productivity metric is wrong in a known way (lines of code, acceptance rate, self-reported speedup); defect escape rate is the one that matters and the slowest to see. Keep your own skills sharp enough to know when the output is wrong.

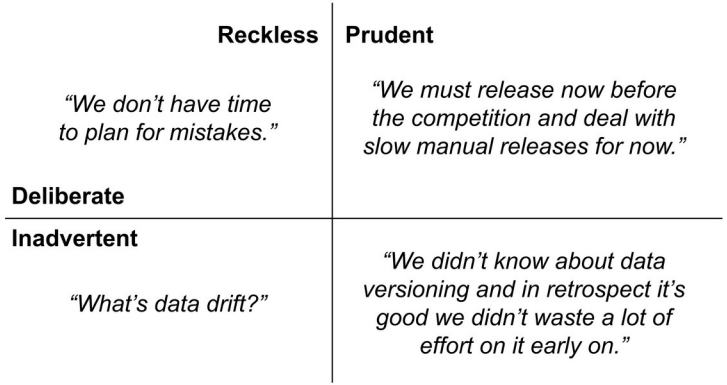
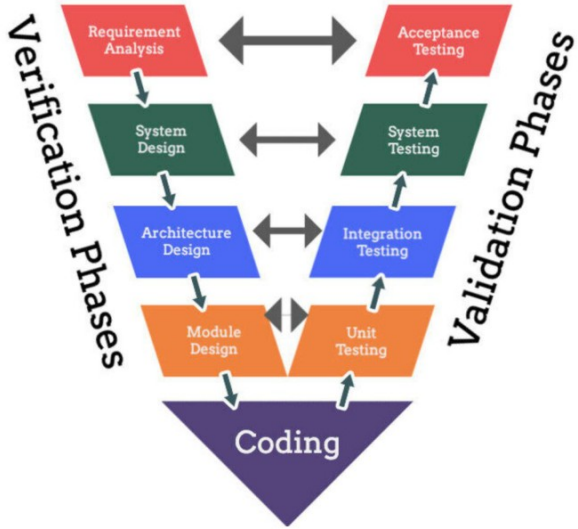


Figure: Kästner 2025, ch. 22, after Fowler. Sculley et al. 2015.

V-model, PR7 and PR8: the process answer

Follow the V; document at every level

- **PR7**: layered decomposition to subsystems and components (ML, non-ML, data, infra); continuous V&V at every level; continuous risk management for the uncertainty of ML; responsibility boundaries for engineers, ML scientists, product managers, data engineers.
- **PR8**: documentation at system, subsystem, and component level, approved by the owners, tracked for every change, with terminology all roles share. In the interviews: decisions that were not written down could not be understood later.
- Gains: clear system boundaries, consistent V&V. Costs: less agility, lower speed to market, everyone has to keep writing.
- Conclusion of the study: the V-model's three characteristics, system decomposition, clear system boundary, consistency of V&V, line up with the collaboration challenges practitioners report. Open question, and a Project 2 direction: a hybrid Agile/V process, or LLM tooling that makes the documentation and V&V cheap enough to keep.



Wu, An Exploratory Study of V-Model in Building ML-Enabled Software, CAIN 2024.



Responsible engineering: six facets, one obligation

Not ethics as an add-on. Properties you design for, like latency

Facet	The question
Fairness	does it work equally well for everyone it affects?
Explainability	can anyone say why this answer?
Safety	what happens when it is wrong in the worst way?
Security & privacy	who can attack it, and what leaks?
Transparency	do people know a model was involved?
Accountability	who answers for it, and how does someone appeal?



Fairness is not one thing, and it is a system concern

The definitions conflict. You must choose, justify the choice, and measure per slice

- Anti-classification (do not use the protected attribute) is weak: proxies leak, zip code encodes race.
- Independence (equal positive rates across groups) and separation (equal error rates across groups) cannot both hold except in degenerate cases. "We made it fair" says nothing until you say which, and why.
- Measuring fairness is Lecture 5's slicing with slices chosen for a legal and moral reason. Durable fixes are in data collection and process: who is missing from the data, can the user contest the decision, is there a monitoring signal?

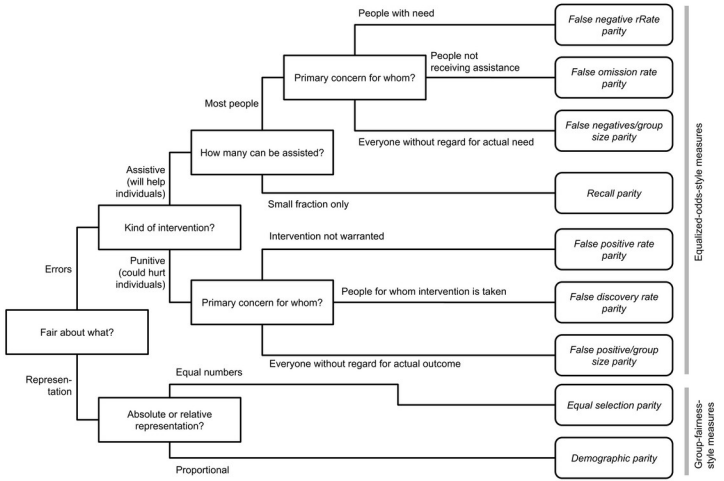


Figure: Kästner 2025, ch. 26. Chouldechova 2017; Kleinberg, Mullainathan & Raghavan 2017.

Explainability, safety, and security

Three distinctions to keep straight

- **Explainability**: global (what has the model learned?) vs local (why this answer?). Post-hoc explanations can be plausible and wrong; an explanation that sounds reasonable raises trust whether or not it is faithful. An inherently interpretable model beats an explained opaque one.
- **Reliability vs safety**: reliable = does the right thing when nothing attacks it; safe = when it does the wrong thing, nobody is harmed. A system can be highly reliable and deeply unsafe.
- **Security**: ML-specific attacks with no analogue in ordinary software: poisoning (corrupt the training data), evasion (a crafted input, like stickers on a stop sign), extraction and leakage (queries recover the model or its data). Prompt injection is the version agents meet.



Figure: Kästner 2025, ch. 27 (Eykholt et al. 2018). Ribeiro et al. 2016; Rudin 2019; Carlini et al. 2021.

Transparency and accountability, and the coding view

The questions asked after something goes wrong

- **Disclosed?** did they know a model was involved. **Explained?** why this answer. **Appealable?** can a human review it. **Accountable?** who answers for it.
- The appeal path is a product feature that must be built, staffed, and tested; the EU AI Act is turning these from good practice into requirements.
- For code AI: does it work beyond Python and English; what licence covers what it memorised; prompt injection through a README; is generated code marked as such in review; and when generated code causes an outage, **the human who merged it owns it**.

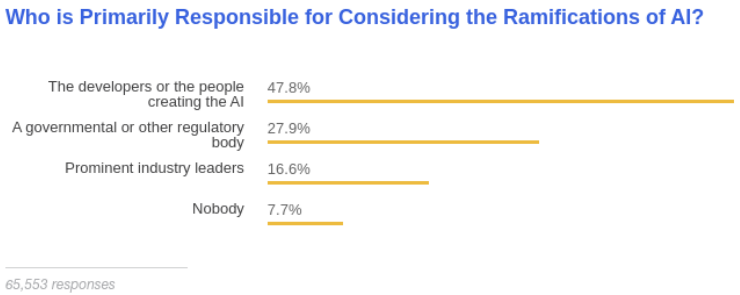


Figure: Kästner 2025, ch. 29. Mitchell et al. 2019.

Module 2 in one line

Model → System → Practice

- Module 1: the model · Module 2: the system · Module 3: the practice · Project 2: your paper.
- The eight lectures: the small box · fit and goals · requirements and mistakes · architecture and skew · four-level QA · production and provenance · process and debt · responsibility.
- Every lecture had the same shape: the model is the small part, and the engineering around it is where products succeed or fail.



Midterm 2: Wednesday, in class

Cumulative: 30 points before Midterm 1, 70 points since

- 50 minutes, closed book, no calculator needed; the few numbers you need are given on the paper.
- Format: 25 multiple choice (2 pts) · 20 true/false (1 pt) · 2 matching tables (10 pts each) · 1 short answer (10 pts) · 1 bonus (10 pts, the hard one). Conceptual; no work-it-out problems.
- 70 points on everything since Midterm 1: the Week 7–8 basics (RAG, the agent loop, evaluation and contamination) and the eight Module 2 lectures, from the small box through today.
- 30 points on the Midterm 1 material: tokens, attention, pretraining and scaling, post-training, pass@k.
- Not examined: the ⚙️ pairings (specs, assuring generated code, productivity).
- How to prepare: the review sheet (on the website) and each deck's Key takeaways slide. The exam questions are close to the review sheet's practice problems, but shorter.



Rapid check: one question per Module 2 lecture

Answer in your head; we go through them together

1. Why is "the model is a small box" the thesis of the module? Name three boxes around it.
2. Give the five conditions under which ML is the right tool, and the four layers of goals.
3. Four kinds of requirements; six patterns for designing for failure; what a fault tree starts from.
4. Five quality attributes that decide an architecture; why training/serving skew is silent; what MLOps adds to DevOps.
5. Four levels of QA; why a model can only be validated; what behavioural tests check.
6. Four safe ways to ship; the two rules of an A/B test; four things to version.
7. Five kinds of ML technical debt; what CACE implies for testing.
8. Three fairness definitions and why they conflict; reliability vs safety; the four accountability questions.



Rapid check: Weeks 1–8 in five questions

Questions 1–3 are the 30 points before Midterm 1; 4–5 count with Module 2

1. Tokens and attention: what does the causal mask do, and why does softmax need the scale factor?
2. Pretraining: what does the loss measure; what does the Chinchilla rule say about tokens and parameters?
3. Post-training: SFT, RLHF, and DPO: what signal does each learn from?
4. Evaluation: pass@k from n samples and c passes; what contamination is and one way to detect it.
5. Retrieval and agents: the four steps of RAG; the agent loop and why a verifier matters.



Before Wednesday

- Read the review sheet; do the practice problems we did not get to today.
- The review sheet is all you need; the Module 2 worksheet on the website is optional extra practice.
- Bring a pen. No calculator needed.
- After the exam: Friday is the ⚙️ agentic SE deep dive; Project 2 drafts are due Fri Dec 4.

